

approvals.

A multi-tenant expense approval tool for small teams

72-hour take-home submission

NEXT.JS · SUPABASE (POSTGRES + RLS) · VERCEL · SENTRY

approvals.

WORKSPACE

- Dashboard
- My requests
- New request

REVIEW

- Approval queue 3
- All requests

AUDIT

- Activity

Acme Inc / Approval queue

Approver  BA 

REVIEW

Approval queue

3 requests awaiting review, oldest first. Items marked "admin" are above your limit.

REQUEST	REQUESTER	AMOUNT	STATUS	UPDATED	
Figma subscription Software	Carol Requester	\$144.00	Pending	1h ago	<button>Review →</button>
Monitor arm Equipment	Dan Requester	\$1,500.00	ADMIN Pending	1h ago	<button>Review →</button>
Annual offsite Travel	Alice Admin	\$5,000.00	ADMIN Pending	1h ago	<button>Review →</button>

The problem

02 / 15

Small teams approve money over email and chat — no trail, no accountability, no status. In fintech, an approval with no immutable record is a compliance failure, not an inconvenience.

BEFORE — EMAIL & CHAT

expenses · Slack

Dana

can someone approve my \$1,500 laptop? 🙏

You

who approves these now that Sam left?

Priya

i think i ok'd it last week? or was that the monitor

Dana

did this get approved?? finance is asking 🤔

You

no idea — check the email thread

AFTER — ONE AUDITED RECORD

approvals.

Acme Inc / Request

Admin

WORKSPACE

Dashboard

My requests

New request

REVIEW

Approval queue

All requests

ADMINISTRATION

Members

Settings

AUDIT

Activity

Acme Inc

ADMIN

BACK TO REQUESTS

EQUIPMENT

New developer laptop

requested by Carol Requester · 2h ago

\$1,200.00

ABOVE THRESHOLD · ADMIN REQUIRED

Current one is dying

Approved

by Alice Admin · 33m ago

"Approved by admin (over limit)"

HISTORY

Carol Requester submitted this request

2h ago

Alice Admin approved this request

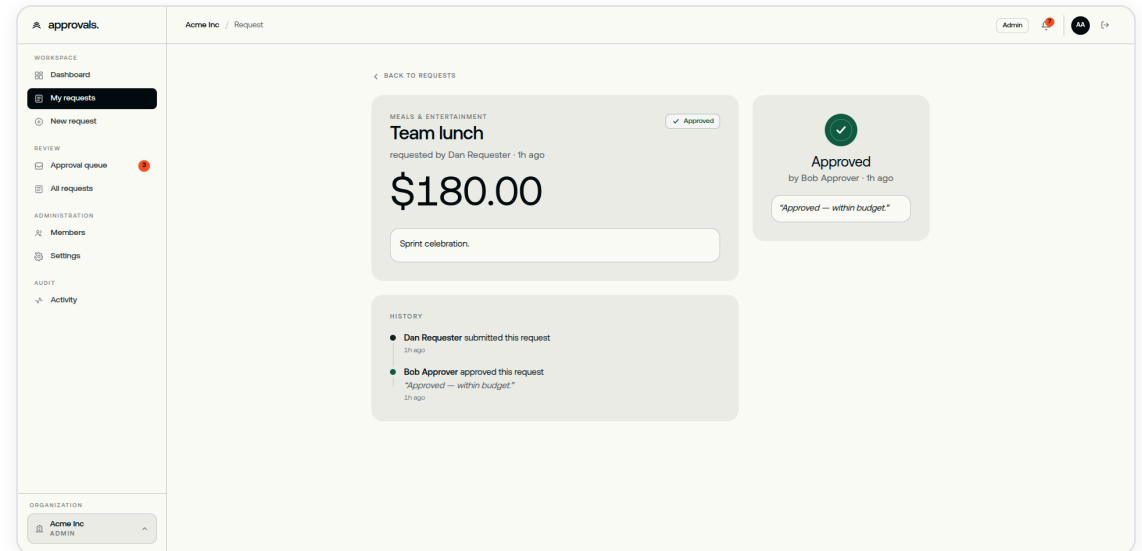
"Approved by admin (over limit)"

33m ago

What I built

03 / 15

- Multi-tenant expense approval app: request → review → decision
- Three roles: **requester**, **approver**, **admin** (admins can also submit)
- Full lifecycle: **pending** → **approved** / **rejected** / **withdrawn**
- Resubmission creates a **new** request — the rejection stays on record permanently
- Immutable audit trail on every state change



REQUEST DETAIL — EVENT TIMELINE (ADMIN)

Data model

04 / 15

- Six core tables: **organizations** · **profiles** · **memberships** · **requests** · **request_events** · **invitations**
(plus a notifications table for in-app alerts)
- **memberships** is the user ↔ org join — it carries the role
- Every domain table is scoped by **org_id** — tenant isolation at the data layer
- Money as **integer minor units (cents)** — never floats
- **request_events** is append-only — no UPDATE/DELETE policy exists

```
create table requests (  
  id          uuid primary key,  
  org_id      uuid not null references organizations,  
  requester_id uuid not null references auth.users,  
  amount_minor bigint not null check (amount_minor > 0), -- cents  
  currency    text not null,  
  status      request_status default 'pending'  
);  
  
create table request_events ( -- append-only  
  id          uuid primary key,  
  org_id      uuid not null references organizations,  
  request_id  uuid not null references requests,  
  actor_id    uuid references auth.users,  
  from_status request_status,  
  to_status   request_status,  
  note        text,  
  created_at  timestampz default now()  
);
```

Excerpt — ..._init.sql

Security architecture

05 / 15

- Auth via **@supabase/ssr httpOnly cookies** — JWT never in localStorage
- **RLS on every table**, default-deny
- Policies written **per operation** — SELECT, INSERT, UPDATE, DELETE separately
- A SELECT policy grants **no** write access — each verb explicitly allowed or denied
- Helpers **is_org_member / is_org_approver / is_org_admin** keep policies clean
- Service-role key is **server-only** — the browser holds the public anon key; RLS protects the data

```
-- SELECT: owner OR an approver in the org
create policy requests_select on requests
  for select to authenticated
  using ( requester_id = auth.uid()
         or is_org_approver(org_id) );

-- INSERT: only for yourself, only in your org
create policy requests_insert on requests
  for insert to authenticated
  with check ( requester_id = auth.uid()
               and is_org_member(org_id) );

-- No UPDATE / DELETE policy exists.
-- authenticated isn't even granted UPDATE –
-- status changes are impossible outside the RPC.
```

Excerpt — RLS policies on requests

The decision RPC

06 / 15

- **decide_request()** is a **SECURITY DEFINER** Postgres function
- One atomic transaction enforces: member with approver/admin role; **not the requester**; still **pending** (row locked); threshold routing; status flip + audit insert — both or neither
- Business logic lives in the **database**, not the API
- Can't be bypassed via UI or raw API — **authenticated** isn't granted UPDATE on requests

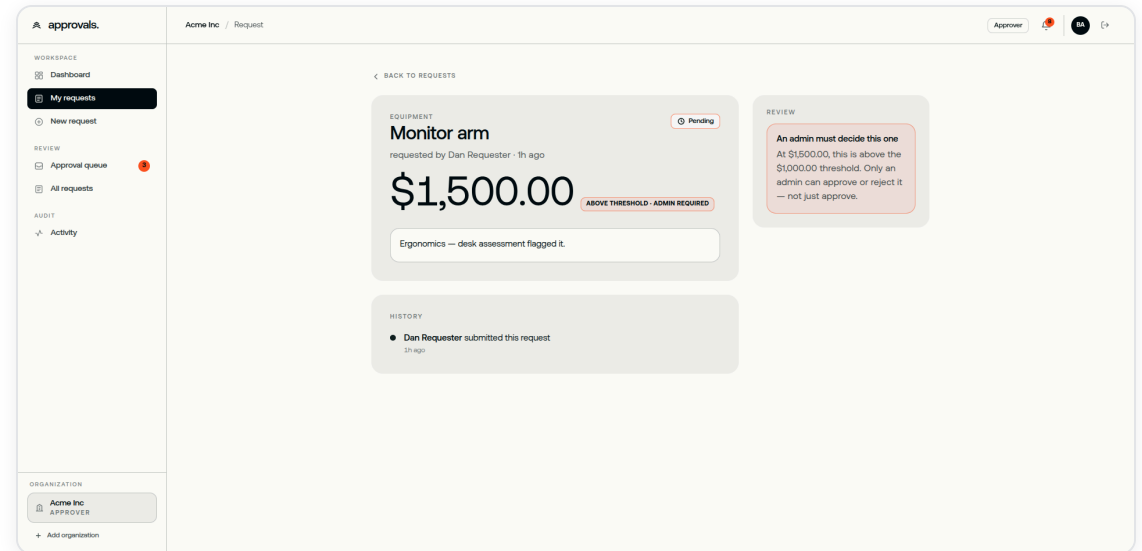
```
create function decide_request(p_request uuid,  
                               p_decision request_status, p_note text)  
security definer set search_path = '' as $$  
  select * into v_req from requests  
  where id = p_request for update;           -- lock  
  if v_req.status <> 'pending' then raise; -- 1 winner  
  if v_req.requester_id = v_uid then         -- segregation  
    raise 'you cannot decide your own request';  
  if not is_org_approver(v_req.org_id) then raise;  
  if v_req.amount_minor > v_threshold       -- threshold  
    and not is_org_admin(org_id) then raise;  
  update requests set status = p_decision ...; -- flip  
  insert into request_events ...;             -- + audit  
$$;
```

Excerpt — decide_request (abridged)

Threshold routing

07 / 15

- Each org sets an **approval_threshold** (in minor units)
- **Under** threshold → any approver or admin may decide
- **Over** threshold → admin role required
- Enforced inside the RPC — the UI flag is informational only
- **Single-admin edge case:** if the only eligible admin is also the requester, it falls back to any approver rather than deadlocking — and the UI says so, never silently
- **Self-approval is never permitted** — regardless of role or threshold

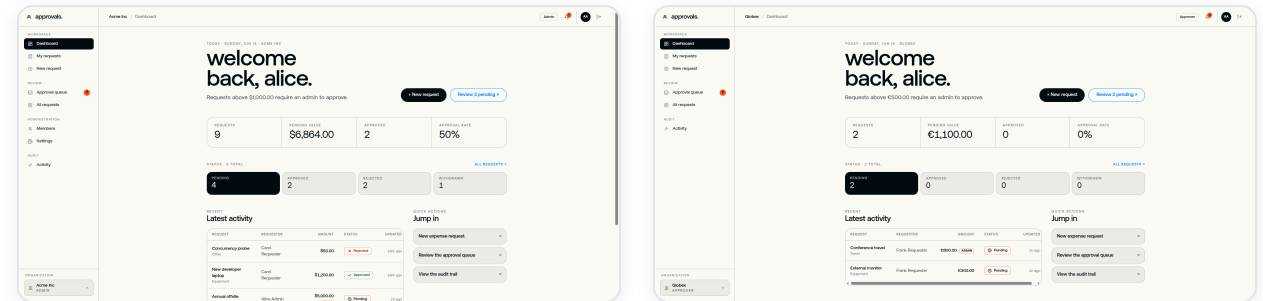


OVER-THRESHOLD ITEM BLOCKED FOR AN APPROVER

Multi-tenancy

08 / 15

- A user can belong to **multiple orgs**, with a different role in each
- The active org is the **current_org** cookie — single source of truth
- Every list query filters by the active **org_id**
- RLS **double-locks** at the database — a direct API call can't return another tenant's rows
- Verified with a red-team script using the public anon key: **9/9** checks pass

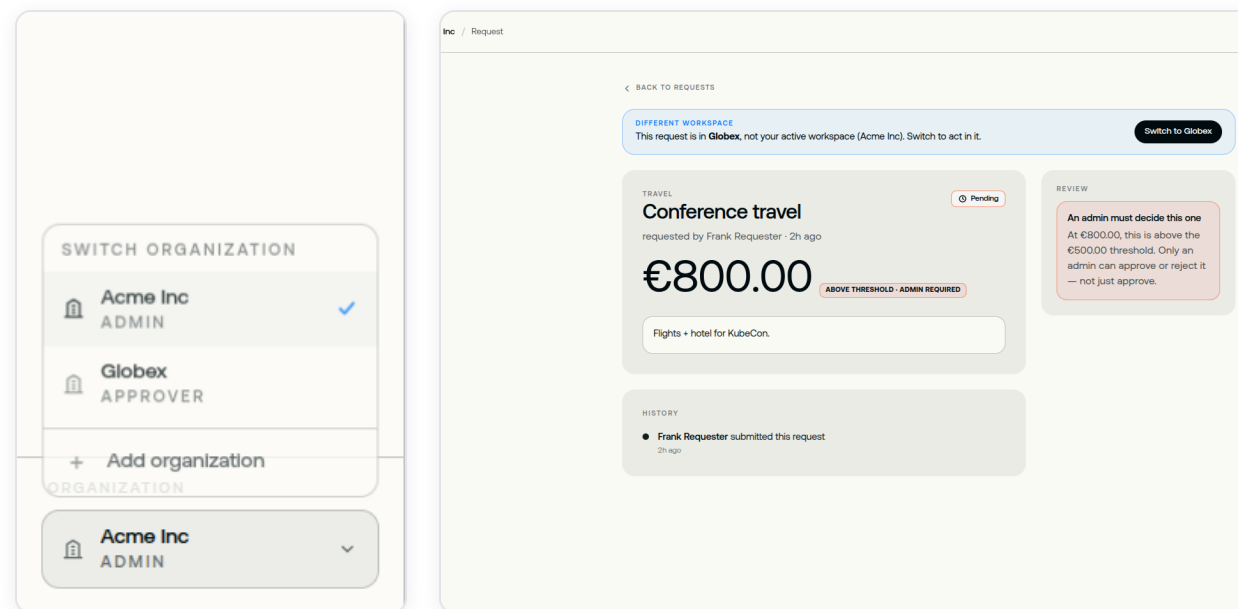


SAME USER, TWO ORGS — ACME (USD) VS GLOBEX (EUR), FULLY SEPARATE DATA

Joining & switching orgs

09 / 15

- Onboarding offers two paths: **create an organization** (you become its admin) or **join with an invite code**
- Invites are admin-issued, role-scoped codes; **accept_invitation()** verifies the caller's email matches
- The **org switcher** re-renders the entire app in the chosen tenant — queue, members, settings, notifications
- A cross-org deep link shows a **switch banner** instead of silently blending context — switching is always deliberate

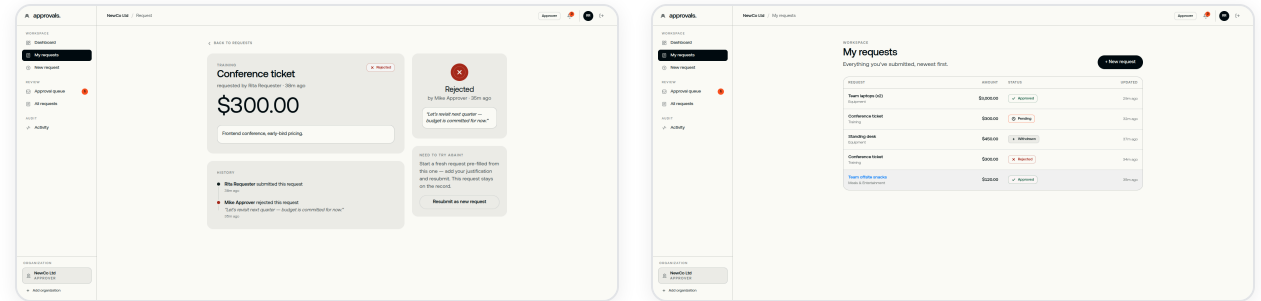


ORG SWITCHER MENU · CROSS-ORG “SWITCH” BANNER

Audit trail & resubmission

10 / 15

- **request_events** is append-only — no UPDATE/DELETE policy exists
- Every decision records **actor, from_status, to_status, note, timestamp**
- A rejected request can be **resubmitted** — it creates a **new** request; the original rejection stays on record permanently
- Both records coexist — nothing is ever overwritten or deleted



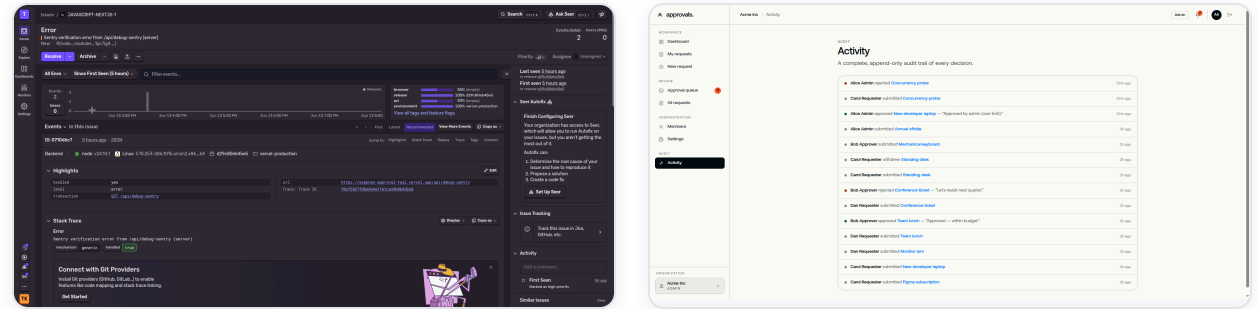
REJECTION + RESUBMIT AFFORDANCE (LEFT) · ORIGINAL AND NEW REQUEST, BOTH ON RECORD (RIGHT)

Observability

11 / 15

- Two layers: **Sentry** for infrastructure errors, the **activity feed** for domain decisions
- Sentry on both **server and client** — errors captured with stack trace + context
- A correlation **request_id** on every server action for log tracing
- **/api/health** returns DB connectivity + latency; **Vercel Web Analytics** enabled
- The activity feed is the **domain audit log** — every decision traceable to actor + timestamp

```
GET /api/health → 200
{ "status":"ok", "db":"up", "latency_ms":16 }
```



SENTRY — A CAPTURED SERVER ERROR · IN-APP ACTIVITY FEED — DOMAIN AUDIT LOG

What I left out — and why

12 / 15

- **Email notifications** — Resend needs a verified custom domain, which a *.vercel.app preview URL can't have.
Integration point stubbed and ready; replaced with in-app notifications. ~30-min add with a domain.
- **Multi-level approval tiers** — a single threshold covers the MVP.
Dual approval (two independent approvers for very large amounts) is the natural next tier; the RPC is where it goes.
- **Native mobile** — responsive to 768px.
Browser-first is correct for an internal ops tool.
- **These were deliberate scoping calls, not oversights.**

What I'd build next

13 / 15

Dual approval

A second required actor **inside the existing** `decide_request` RPC: above a higher tier, the status flip waits for two distinct approvers — same transaction, same audit log, no new surface.

Delegated approval

A `delegations` row checked inside `is_org_approver()`, so an out-of-office approver's authority temporarily passes to a delegate — enforced in RLS, not the UI.

THEN

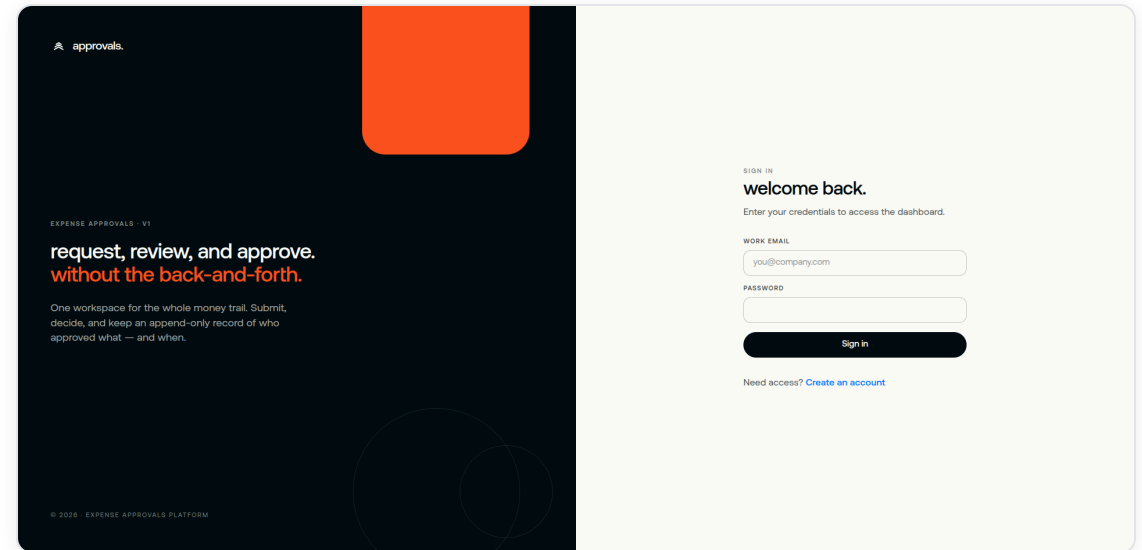
Email + in-app push on a custom domain · **CSV / PDF export** for accounting · **org spending analytics** (by category, month, requester) · **webhooks** for accounting-tool integration

Test accounts

14 / 15

- All passwords: **Password123!**
- **Requester** — requester@acme.test
- **Approver** — approver@acme.test
- **Admin** — admin@acme.test
- Org: **Acme Inc** (USD, \$1,000 threshold) — all three share one org, so the full flow is testable without switching

These are **seeded demo accounts** on a non-routable .test domain — password-reset and other email flows won't deliver. Sign in directly with the credentials above.

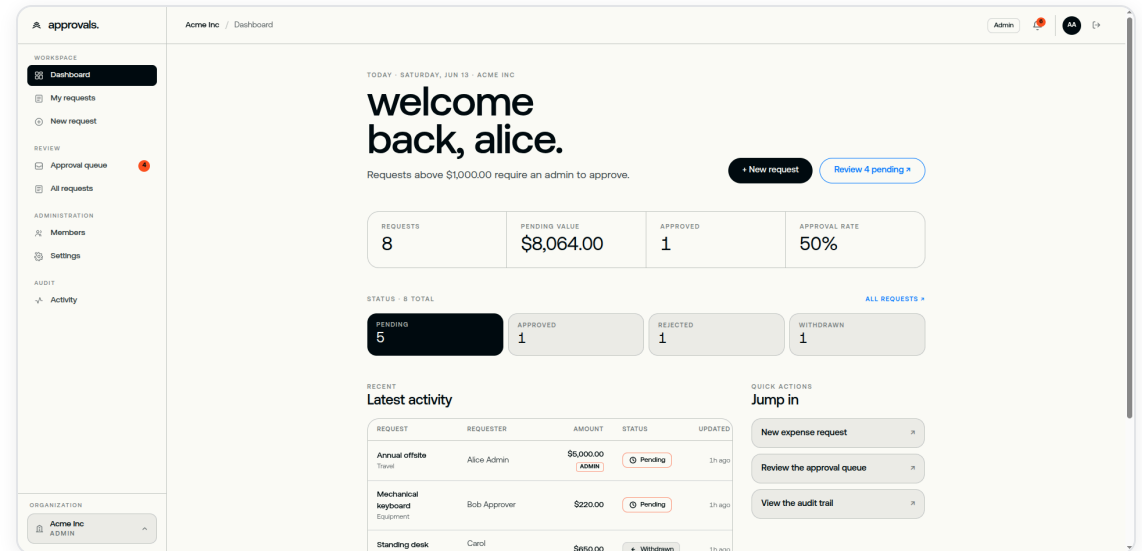


LIVE SIGN-IN

Live app

15 / 15

- expense-approval-teal.vercel.app
(URL also in the submission form — open in incognito to start fresh)
- **Walk the threshold flow in 4 steps:**
 - 1 — as requester@acme.test, submit a **\$1,500** request (over the \$1,000 limit → auto-flagged “**admin required**”)
 - 2 — as approver@acme.test, open it — the decision is **blocked**
 - 3 — as admin@acme.test, **approve** it
 - 4 — reopen the request to see the **event timeline**



LIVE DASHBOARD — REAL DATA